



Plataforma de Gestión Integral de Proyectos y Tareas para Equipos de Desarrollo



Contenido

	Título del Proyecto.....	3
	Visión del Producto	3
	Fase 1: Backend + API Design + Base de Datos	3
♦	Sprint 1: Modelado de Datos y Setup Inicial	3
♦	Sprint 2: Autenticación y Autorización	3
♦	Sprint 3: Gestión de Proyectos.....	4
♦	Sprint 4: Gestión de Tareas.....	4
♦	Sprint 5: Integración y Testing	4
	Fase 2: Frontend + DevOps + Despliegue.....	5
♦	Sprint 6: Setup Frontend y Autenticación	5
♦	Sprint 7: Visualización de Proyectos	5
♦	Sprint 8: Gestión de Tareas.....	5
♦	Sprint 9: Contenedorización y CI/CD	5
♦	Sprint 10: Despliegue en Kubernetes	6
	Tecnología involucrada por Funcionalidad.....	6
	Rúbrica de Evaluación del Proyecto	6
	Escala de Evaluación.....	7

Título del Proyecto

Plataforma de Gestión Integral de Proyectos y Tareas para Equipos de Desarrollo

Visión del Producto

El Product Owner (PO) desea una plataforma que permita a equipos de desarrollo gestionar sus proyectos, tareas, usuarios y flujos de trabajo. Debe incluir funcionalidades como autenticación, gestión de proyectos, asignación de tareas, visualización de estados, y despliegue automatizado. El PO espera entregas incrementales cada dos semanas (Sprints), con funcionalidades completas y testeadas.

Fase 1: Backend + API Design + Base de Datos

◇ Sprint 1: Modelado de Datos y Setup Inicial

En esta primera entrega, el equipo debe definir la estructura de datos que sustentará toda la aplicación. Se espera un modelo entidad-relación que contemple usuarios, roles, proyectos y tareas. El Product Owner necesita validar que la base de datos refleje correctamente las relaciones entre entidades antes de comenzar el desarrollo. Se inicia el repositorio Git con estructura base y se aplican estrategias de branching como GitHubFlow.

Funcionalidad: Diseño del modelo entidad-relación para usuarios, proyectos, tareas, roles.

Tecnologías: MySQL/PostgreSQL, diagramas ER, Git (GitHubFlow).

Objetivo: Tener el modelo de datos aprobado por el PO y repositorio inicial con estructura base.

◇ Sprint 2: Autenticación y Autorización

El sistema debe permitir que los usuarios se registren, inicien sesión y accedan según su rol. Esta funcionalidad es crítica para garantizar la seguridad y personalización de la experiencia. El PO espera un sistema robusto con tokens JWT y roles diferenciados (admin, usuario). Esta entrega sienta las bases para el control de acceso en futuras funcionalidades.



Funcionalidad: Registro, login, gestión de roles (admin, usuario).

Tecnologías: Java, Spring Security, JWT.

Objetivo: Backend funcional con endpoints de autenticación y pruebas unitarias.

◇ Sprint 3: Gestión de Proyectos

Los usuarios deben poder crear y gestionar proyectos. Cada proyecto puede tener múltiples tareas y miembros asignados. El PO quiere ver una API REST clara y funcional que permita CRUD completo sobre proyectos. Esta funcionalidad es el núcleo organizativo de la plataforma.

Funcionalidad: Crear, editar, eliminar proyectos.

Tecnologías: Spring Boot (REST), JPA/Hibernate.

Objetivo: API REST para proyectos con documentación Swagger.

◇ Sprint 4: Gestión de Tareas

Las tareas son el componente operativo del sistema. Deben poder crearse, asignarse a usuarios, y cambiar de estado (To Do, In Progress, Done). El PO espera una API bien estructurada, validada con Spectral, y diseñada también en MuleSoft para facilitar su consumo por otros sistemas. Esta entrega conecta directamente con la lógica de trabajo en equipo.

Funcionalidad: CRUD de tareas, asignación a usuarios, estados (To Do, In Progress, Done).

Tecnologías: Spring Boot, OpenAPI, Spectral (validación sintáctica).

Objetivo: API REST validada con Spectral y diseño en MuleSoft API Designer.

◇ Sprint 5: Integración y Testing

Antes de pasar al frontend, el PO necesita asegurarse de que el backend está completamente probado y documentado. Se deben realizar pruebas unitarias e integración, y dejar lista la documentación OpenAPI. Esta entrega garantiza la calidad y estabilidad del backend para su consumo por el frontend.

Funcionalidad: Pruebas de integración, documentación completa de APIs.

Tecnologías: JUnit, Postman, MuleSoft (API Console).

Objetivo: Backend listo para ser consumido por el frontend.



Fase 2: Frontend + DevOps + Despliegue

◇ Sprint 6: Setup Frontend y Autenticación

El frontend debe permitir a los usuarios registrarse e iniciar sesión. Esta funcionalidad debe estar conectada al backend y manejar correctamente los tokens JWT. El PO quiere ver una interfaz funcional y segura que refleje los roles definidos en el sistema.

Funcionalidad: Login y registro desde interfaz web.

Tecnologías: React o Angular, Axios, JWT.

Objetivo: Frontend conectado al backend con autenticación funcional.

◇ Sprint 7: Visualización de Proyectos

Los usuarios deben poder ver sus proyectos, crear nuevos y editarlos desde la interfaz. El PO espera una experiencia fluida y visualmente clara, que permita gestionar proyectos sin necesidad de acceder al backend directamente. Esta entrega mejora la usabilidad y accesibilidad del sistema.

Funcionalidad: Listado de proyectos, creación y edición desde UI.

Tecnologías: React/Angular, Material UI/Bootstrap.

Objetivo: Interfaz amigable para gestión de proyectos.

◇ Sprint 8: Gestión de Tareas

Se implementa un tablero tipo Kanban para visualizar y gestionar tareas. El PO quiere una interfaz dinámica que permita arrastrar tareas entre estados y editar su contenido. Esta funcionalidad es clave para la productividad del equipo y la visualización del flujo de trabajo.

Funcionalidad: Visualización tipo Kanban, edición de tareas, cambio de estado.

Tecnologías: React DnD / Angular CDK, consumo de API REST.

Objetivo: Tablero funcional con interacción dinámica.

◇ Sprint 9: Contenedorización y CI/CD

El sistema debe estar preparado para ser desplegado automáticamente. Se dockeriza el backend y frontend, y se configura un pipeline de integración continua. El PO espera que cada cambio en el código se pruebe y despliegue automáticamente en un entorno controlado.

Funcionalidad: Dockerización de backend y frontend, pipeline de integración continua.

Tecnologías: Docker, GitHub Actions, Helm.

Objetivo: Imágenes Docker listas y pipeline CI configurado.

◇ Sprint 10: Despliegue en Kubernetes

Finalmente, el sistema se despliega en un clúster Kubernetes usando ArgoCD. El PO quiere ver el producto funcionando en un entorno real, con gestión de versiones y despliegue automatizado. Esta entrega representa la culminación del proyecto y su puesta en producción.

Funcionalidad: Despliegue automático en clúster con ArgoCD.

Tecnologías: Kubernetes, ArgoCD, Helm Charts.

Objetivo: Aplicación desplegada y accesible desde entorno cloud/local.

Tecnología involucrada por Funcionalidad

Funcionalidad	Descripción	Tecnologías Impactadas
Autenticación	Permite acceso seguro a la plataforma	Spring Security, JWT, React/Angular
Gestión de Proyectos	CRUD de proyectos con relación a usuarios	Spring Boot, JPA, React/Angular
Gestión de Tareas	CRUD de tareas con estados y asignación	Spring Boot, React DnD, Angular CDK
API REST	Comunicación entre frontend y backend	OpenAPI, Spectral, MuleSoft
CI/CD	Automatización de pruebas y despliegue	GitHub Actions, Docker, Helm
Despliegue	Orquestación y despliegue en clúster	Kubernetes, ArgoCD

Rúbrica de Evaluación del Proyecto

Categoría	Criterio	Indicadores de Evaluación	Puntuación Máxima
Planificación y trabajo en equipo	Organización del trabajo por Sprints	Uso de herramientas ágiles (GitHub Projects, tableros Kanban, reuniones Scrum)	10 pts
	Colaboración y comunicación	Participación activa, resolución de conflictos, documentación compartida	10 pts
Backend y API	Modelado de datos	Correcta estructura relacional, normalización, uso de claves	10 pts



	Desarrollo de endpoints	Funcionalidad completa, seguridad, validación	10 pts
	Documentación de APIs	Uso de Swagger/OpenAPI, Spectral, MuleSoft	10 pts
Frontend	Interfaz de usuario	Usabilidad, diseño responsive, navegación clara	10 pts
	Consumo de APIs	Integración correcta con backend, manejo de errores	10 pts
	Interacción dinámica	Drag & drop, actualizaciones en tiempo real	10 pts
DevOps y despliegue	Contenedorización	Uso de Docker, estructura de imágenes	5 pts
	CI/CD	Pipeline funcional con GitHub Actions	5 pts
	Despliegue en Kubernetes	Uso de Helm, ArgoCD, despliegue automatizado	10 pts
Documentación y presentación final	Informe técnico	Claridad, estructura, justificación de decisiones	5 pts
	Presentación oral	Comunicación, defensa del trabajo, uso de recursos visuales	5 pts

Escala de Evaluación

Excelente (90-120 pts): El proyecto cumple con todos los requisitos funcionales, está bien documentado y demuestra dominio técnico y trabajo colaborativo.

Notable (70-89 pts): El proyecto está completo, aunque presenta pequeñas carencias en integración, documentación o despliegue.

Aprobado (50-69 pts): El proyecto funciona parcialmente, con entregables básicos, pero sin profundidad técnica o integración completa.

Insuficiente (<50 pts): El proyecto no cumple con los requisitos mínimos o no se ha entregado adecuadamente.